Reporte

EQUIPO 4	
García Cerda Randal Guillermo (2112493 ITS) Sarmiento Hernández Cesar Martín (2119427 ITS) Torres Pasillas Angela Susana (2101844 ITS).	
Nombre del curso: Componentes de Lógica de Negocios	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: 4	**Actividad**: Unidad 4.
Fecha: 26/10/2026	
Bibliografia (Formato APA): Notas de clase	

Objetivo:

El estudiante implementará los componentes de lógica de negocio diseñados en U3: entidades DDD, reglas con Specification Pattern, máquinas de estado, Domain Events y APIs REST.

Domain Events y diagrama de estados

1. Qué es un Domain Event

Un Domain Event es un hecho del negocio que ya ocurrió, nombrado en pasado (OrderConfirmed, no ConfirmOrder). En TechStore el Aggregate Root Order registra un evento en su lista events cada vez que completa una transición de estado. Todos heredan de DomainEvent y llevan name, order_id y occurred_at (fecha y hora UTC).

2. Eventos implementados

Evento	Transición	Se registra cuando…	Datos adicionales
OrderConfirmed	DRAFT → CONFIRMED	confirm() valida estado DRAFT, pedido con artículos y cliente activo	—
OrderShipped	CONFIRMED → SHIPPED	ship(tracking_number) valida estado CONFIRMED y guía no vacía	tracking_number
OrderCancelled	CONFIRMED → CANCELLED	cancel(reason) valida estado CONFIRMED y motivo no vacío	reason
OrderDelivered	SHIPPED → DELIVERED	deliver() valida estado SHIPPED	—
OrderReturned	SHIPPED → RETURNED	return_order() valida estado SHIPPED	—

Cada evento es un hecho consumado: no pide que algo se haga, informa que algo ya se hizo. Por eso se nombran en pasado y se crean únicamente cuando la transición de estado del pedido terminó con éxito. A continuación se explica cada uno: qué hecho del negocio representa, qué condiciones deben cumplirse, qué cambia en el pedido y qué información viaja con el evento.

2.1 OrderConfirmed (DRAFT → CONFIRMED)

Qué significa: El cliente y la tienda dejan de negociar el pedido: queda aceptado y listo para prepararse. Es el primer evento de la vida de un pedido y marca el paso de "borrador" a compromiso real.

Cuándo se registra: Se registra al terminar con éxito confirm(). El método comprueba tres cosas: que el pedido esté en estado DRAFT, que contenga al menos un artículo y que el cliente esté activo. Si cualquiera falla, no hay confirmación.

Qué cambia: El estado pasa a CONFIRMED y se agrega OrderConfirmed a la lista events.

Datos adicionales: Ninguno adicional; bastan name, order_id y occurred_at, porque el solo hecho de la confirmación es la información.

Por qué importa: Es el requisito de todo lo demás: solo un pedido confirmado puede enviarse o cancelarse. Un suscriptor podría, por ejemplo, avisar al cliente de que su pedido fue recibido.

Caso relacionado: En E12, O013 se confirma dos veces: la primera genera el evento y la segunda lanza InvalidOrderTransition (HTTP 409) sin duplicarlo.

2.2 OrderShipped (CONFIRMED → SHIPPED)

Qué significa: El paquete salió de la tienda y ya está en manos de la paquetería. Es la prueba de que existe un envío real que puede rastrearse.

Cuándo se registra: Se registra al terminar con éxito ship(tracking_number). Exige que el pedido esté en CONFIRMED y que la guía de rastreo no esté vacía; un envío sin guía no se considera enviado.

Qué cambia: El estado pasa a SHIPPED y se agrega OrderShipped a events.

Datos adicionales: tracking_number: el número de guía. Viaja en el evento para que quien lo reciba pueda informar o rastrear el paquete sin consultar de nuevo el pedido.

Por qué importa: Tras este evento el pedido ya no puede cancelarse: solo puede entregarse o devolverse. Un suscriptor podría enviar al cliente el número de guía.

Caso relacionado: Se registra al final, después de validar. Si se registrara antes y la guía resultara inválida, el sistema anunciaría un envío inexistente (ver sección 5).

2.3 OrderCancelled (CONFIRMED → CANCELLED)

Qué significa: El pedido se anuló antes de salir de la tienda. Es un estado final: el pedido no continúa.

Cuándo se registra: Se registra al terminar con éxito cancel(reason). Exige que el pedido esté en CONFIRMED y que el motivo no esté vacío, de modo que toda cancelación quede justificada.

Qué cambia: El estado pasa a CANCELLED y se agrega OrderCancelled a events.

Datos adicionales: reason: el motivo de la cancelación, útil para auditoría y para saber por qué se pierden pedidos.

Por qué importa: Solo se puede cancelar mientras el pedido está CONFIRMED; una vez enviado ya no. Un suscriptor podría notificar la cancelación al cliente o liberar el inventario reservado.

Caso relacionado: En E13, O014 ya está SHIPPED y cancel(motivo) falla con InvalidOrderTransition (HTTP 409): no se registra OrderCancelled.

2.4 OrderDelivered (SHIPPED → DELIVERED)

Qué significa: El cliente recibió el paquete. Es el cierre exitoso del ciclo del pedido y un estado final.

Cuándo se registra: Se registra al terminar con éxito deliver(). Solo exige que el pedido esté en SHIPPED: no se puede entregar algo que no salió.

Qué cambia: El estado pasa a DELIVERED y se agrega OrderDelivered a events.

Datos adicionales: Ninguno adicional.

Por qué importa: Cierra el pedido sin incidencias. Un suscriptor podría, por ejemplo, pedirle al cliente una reseña del pedido.

Caso relacionado: Es mutuamente excluyente con OrderReturned: desde SHIPPED el pedido toma uno de los dos caminos, nunca ambos.

2.5 OrderReturned (SHIPPED → RETURNED)

Qué significa: El paquete enviado volvió a la tienda sin completarse la entrega. Es un estado final.

Cuándo se registra: Se registra al terminar con éxito return_order(). Solo exige que el pedido esté en SHIPPED. El método se llama return_order y no return porque return es palabra reservada del lenguaje.

Qué cambia: El estado pasa a RETURNED y se agrega OrderReturned a events.

Datos adicionales: Ninguno adicional.

Por qué importa: Permite distinguir un pedido que nunca llegó de uno entregado. Un suscriptor podría iniciar el reembolso o reponer el inventario.

Caso relacionado: No existe devolución desde CONFIRMED ni desde DELIVERED: el diagrama de estados solo la permite desde SHIPPED.

3. Regla de consistencia: validar → cambiar → registrar

Cada método de transición sigue siempre el mismo orden: 1) valida todas las precondiciones; 2) cambia el estado; 3) registra el evento. Si algo falla en el paso 1, se lanza la excepción y el pedido queda exactamente igual, sin evento. Por eso un evento en la lista siempre corresponde a una transición que sí ocurrió.

Ejemplo real: O013 confirmado dos veces (E12)

Paso	Operación	Resultado	Estado	Eventos
0	Pedido recién cargado	—	DRAFT	[]
1	confirm()	Éxito	CONFIRMED	[OrderConfirmed]
2	confirm()	InvalidOrderTransition (HTTP 409)	CONFIRMED	[OrderConfirmed] (sin duplicado)

Ejemplo real: O014 no se cancela después de enviarse (E13)

Paso	Operación	Resultado	Estado	Eventos
1	confirm()	Éxito	CONFIRMED	[OrderConfirmed]
2	ship(guía)	Éxito	SHIPPED	[OrderConfirmed, OrderShipped]
3	cancel(motivo)	InvalidOrderTransition (HTTP 409)	SHIPPED	sin OrderCancelled

4. Diagrama de estados

El diagrama representa exactamente las cinco transiciones implementadas en app/domain.py. DELIVERED, RETURNED y CANCELLED son estados finales. Archivos: Diagrama_Estados_TechStore.png / .pdf / .drawio (editable) / .mmd (Mermaid).

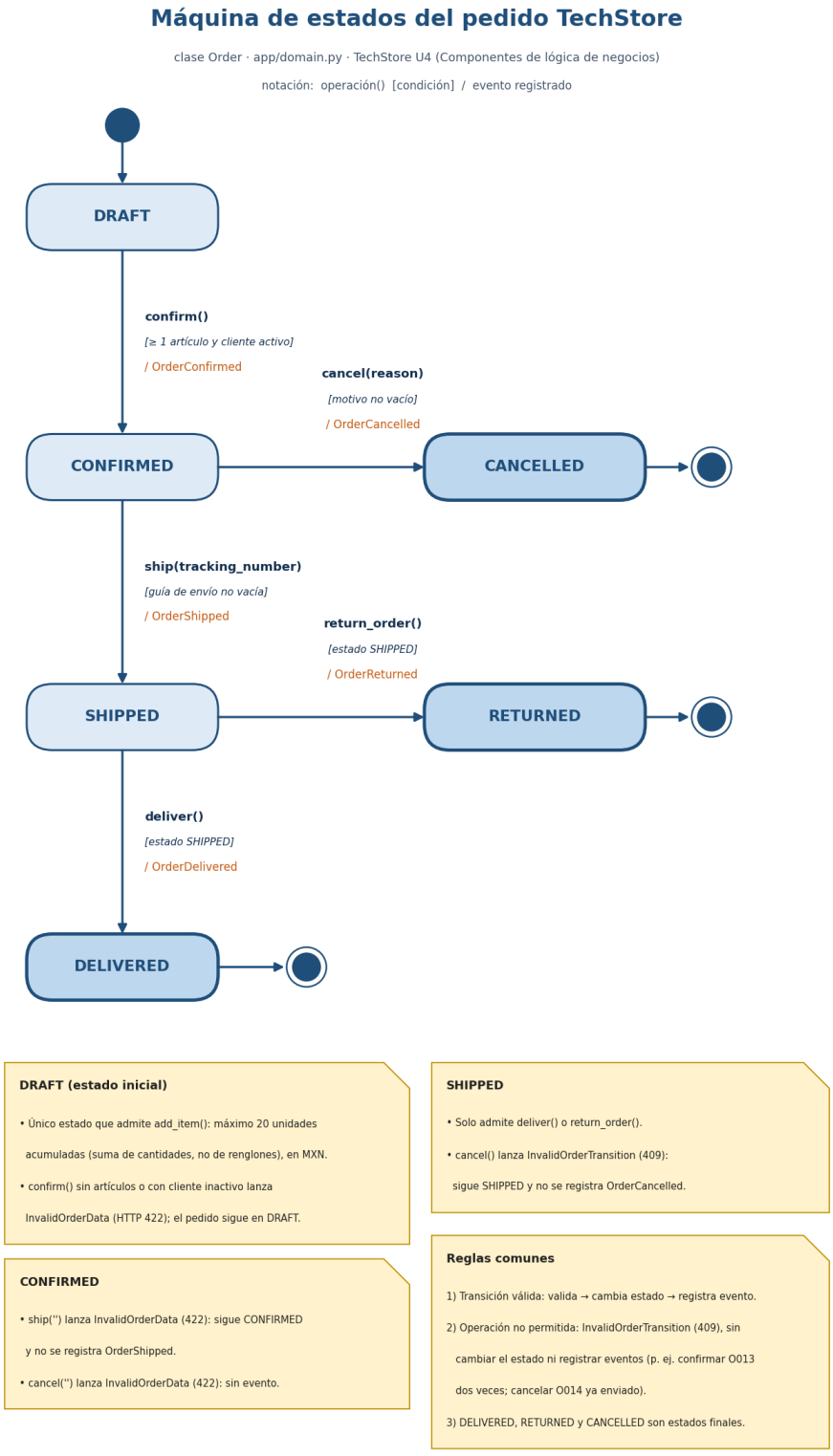


Figura 1. Máquina de estados del pedido.

5. Qué pasaría sin esta regla

Si OrderShipped se registrara antes de validar la guía y la validación fallara, el sistema anunciaría un envío que no existe y un suscriptor podría avisar al cliente de un paquete que nunca salió. Registrar el evento al final evita ese estado inconsistente.